

Rapport parallélisme

Nom: Bastien NGUYEN

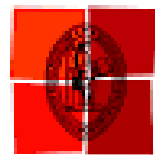
Numéro étudiant: 22300553

Groupe: B11

Exercice 7



UNIVERSITÉ
TOULOUSE III
PAUL SABATIER



Université
de Toulouse

Introduction

Pour rappel j'ai 6 coeur sur ma machine. Tout les résultats que je montre seront fait à partir de ma machine.

Voici la liste de tout les éléments dans mon archive:

- un dossier datasets ou sont stocke les fichier de tests
- un fichier ex7-seq qui est la version séquentielle que j'ai réalisé

- un fichier ex7-par qui est la version parallélisé de la version séquentielle.

j'ai exécuté plusieurs fois ces programmes pour faire une moyenne en temps d'exécution tout en vérifiant bien que les résultats soient cohérents. Je n'ai pas fait de version moyenne cette fois ci car il fallait etre sur que le programme calcule bien k-mean sans faire d'erreur.

Séquentielle

Pour faire cette version j'ai eu beaucoup de mal car ,malgré le fait que je suive l'UE IA, l'algorithme était confu dans ma tete j'ai donc mis beaucoup de temps à l'écrire. Il m'as fallu pour cela regarder le wikipédia et calculer à la main des exécutions de cet algorithme. Après cela j'ai compris que j'avais mal fait ma structure cluster. A la base je voulais y stocker l'entièreté des points appartenant à un cluster dans un tableau comme on le feras dans un calcul sur un papier mais cela m'as posé beaucoup de problème au final j'ai compris que je n'avais théoriquement pas besoin de les stocker mais de juste mettre à jour ses coordonnées et le nombre d'élément qu'il contient. Pour mes mesures temporelles j'ai décidé d'uniquement mesurer mon calcul des cluster car c'est cette partie que je vais paralléliser.

Voici les mesures que j'ai obtenu sur les deux fichiers

10000_4	20000_4
0,093s	11,077s

On remarque que le fichiers contenant 200000 points prend un certain temps d'exécution il seras donc un bon exemple d'utilisation de la parallélisation.

Parallèle

Pour réaliser la version parallèle j'ai décidé de faire en sorte que l'entièreté de mon while (sauf mon `it++` qui me sert de compteur pour forcer l'arrêt) sera en région parallèle. Il faut savoir qu'au début je voulais créer les threads avant de rentrer dans le while. Je pense toujours que c'est possible mais je devais laisser une erreur car cela faussait mes résultats mais je déduis facilement que pour les petits fichiers le fait de recréer plusieurs fois des régions parallèles va peser sur la balance beaucoup plus que sur des grands fichiers.

Les initialisations des sommes et du compteur `count` sont parallélisables avec un `for` classique.

Pour leur calcul la par contre c'est très différent car plusieurs threads peuvent essayer d'augmenter une valeur en même temps et donc créer un problème de concurrence. Pour palier à cela j'ai donc créé plusieurs tableaux de sommes locales et à la fin j'entre en section critique pour ajouter ces sommes locales aux réelles sommes. Grâce à ça je peux donc faire un `for` classique et comme un thread ayant fini n'a pas besoin d'attendre les autres pour commencer à rentrer en section critique j'utilise un `nowait`.

Pour le calcul de la variation comme je n'ai qu'une variable à protéger je peux directement utiliser une réduction dessus c'est plus facile et efficace qu'une "variation locale".

Parallèle

Voici les mesures temporelles que j'ai eu sur cette version:

Nombre de threads	10000	200000
1	0,097s	11,132s
2	0,058s	6,079s
4	0,029s	3,845s
6	0,026s	3,127s
8	0,024s	2,474s
10	0,02s	2,138
12	0,126	2,499

On voit bien qu'augmenter le nombre de threads aide beaucoup. On remarque aussi qu'après avoir dépassé mon nombre de coeur (6) on gagne de moins en moins de temps voir on remonte même à 12 threads

Analyse

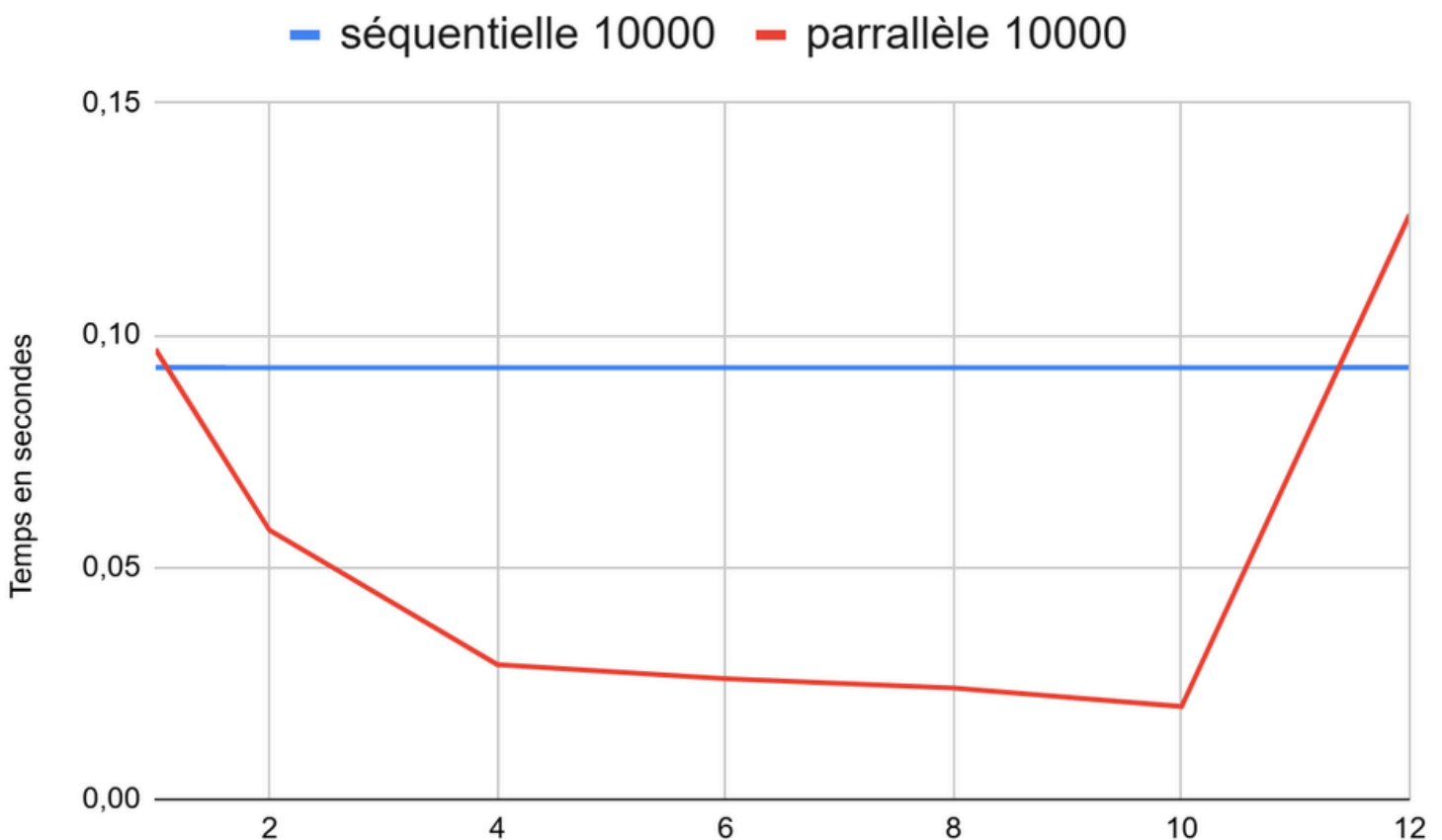
Grace aux mesures des deux versions on peut calculer l'accélération obtenu pour chaque image.

Voici un tableau des différentes accélération obtenue en fonction du nombre de threads.

Nombre de threads	10000	200000
1	0,96	0,99
2	1,6	1,83
4	3,21	2,9
6	3,58	3,56
8	3,88	4,5
10	4,65	5,21
12	0,74	4,45

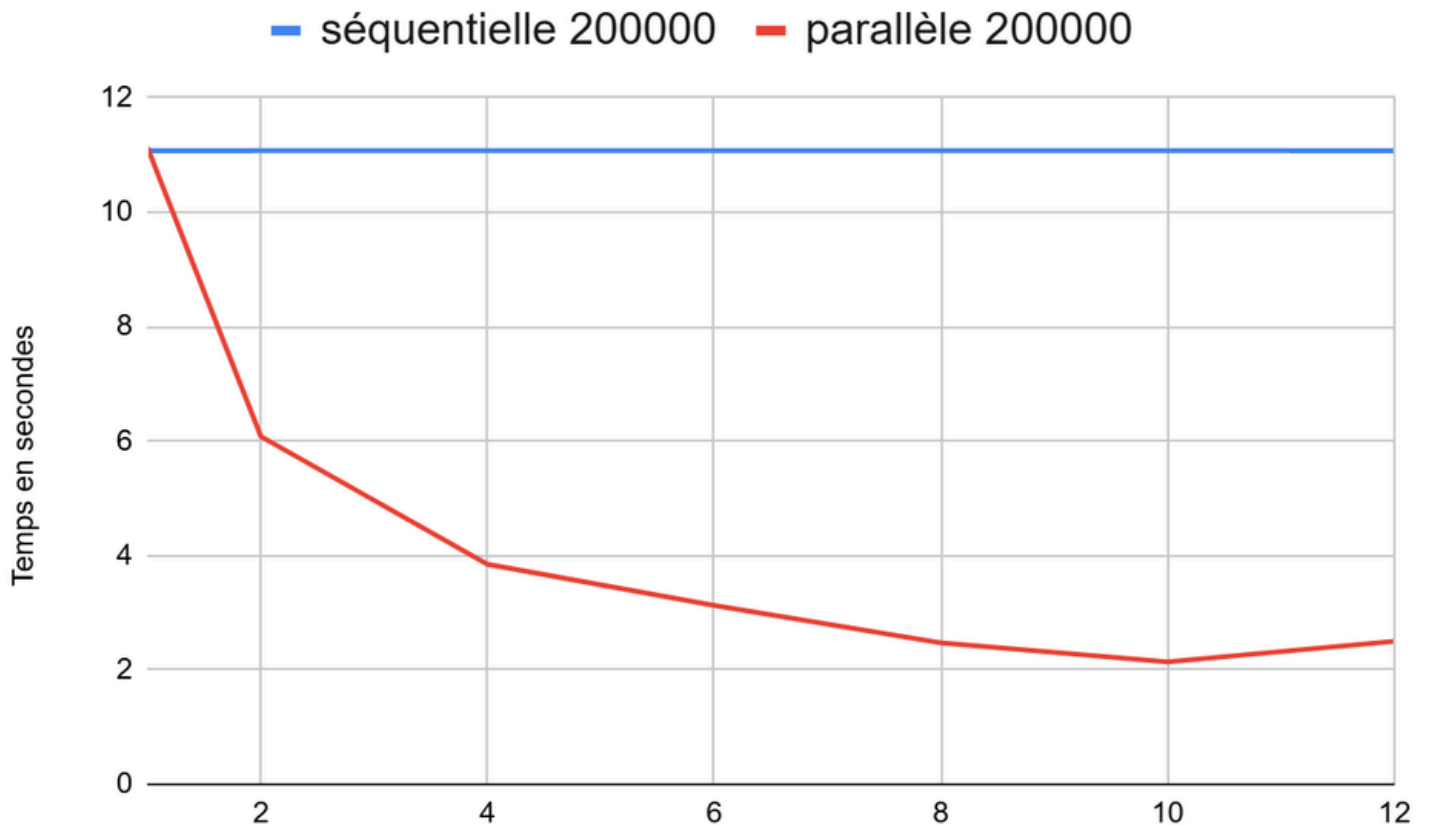
Analyse

En regardant les mesures d'accélération on peut conclure que plus il y a de calcul à faire plus la version parallèle est rentable mais qu'en dépassant mon nombre de coeur l'accélération ralentit puis même baisse avec trop de threads. On observe même que pour 10000 il devient plus rentable de rester en séquentielle que d'utiliser 12 threads. Il faut qu'en même prendre en considération le fait que ma région parallèle est recrée pour chaque boucle while. Cela prouve encore une fois que dépasser le nombre de coeur est une grosse erreur pour des petits fichiers. On peut même modéliser une courbe représentant l'évolution des temps de calculs sur 10000 points



Analyse

Pour 200000, comme prévu la parallélisation est bien plus rentable. On gagne beaucoup de seconde et meme à 12 threads cela reste plus rentable que la version séquentielle. Voici une courbe représentant le temps d'exécution pour 200000 en fonction du nombre de threads. On y voit bien dessus que le cout des primitives openMp influent sur le temps d'exécution.



Conclusion

Ce tp rassemble plusieurs éléments important de la parallélisation. On y voit des sommes local, des réduction des nowait , des section critique et aussi des single.

Le tp montre bien que paralléliser un programme peut etre très rentable sur certain fichier et moins dans d'autre.

Comme la théorie nous le dis dépasser le nombre de coeur est presque inutile voir meme impacte négativement le temps d'exécution.

La plus grande difficulté que j'ai eu fut d'écrire la version séquentielle mais comme je l'ai écrite cela m'as facilité sa paralélisation car j'avais déjà pu réfléchir à ce que je pouvais faire.